

CYBERSENTINEL

AI-Powered Behavioral Anomaly Detection for Cybersecurity

Honeywell Hackathon 2026 | Problem Q4 | Category: Software | Sonali Kumari

PROBLEM STATEMENT (Q4)

"Design an AI/ML system that models normal access behaviour, detects intrusions, classifies anomaly type, and provides explainable risk scores for SOC analysts."

5 Core Challenges:

Sequential data | Class imbalance (<3%)
Concept drift | Explainability | Cold-start

MODELS IMPLEMENTED (Final System)

$Risk = 0.35 \times IF + 0.35 \times LSTM + 0.20 \times XGB + 0.10 \times History$

- 1. Isolation Forest (unsupervised, 200 trees)**
Detects outliers without labels
- 2. LSTM Autoencoder (sequence-aware, 64>32)**
Learns normal temporal patterns, flags deviations
- 3. XGBoost Classifier (supervised, 300 trees)**
Classifies 7 attack types with SMOTE

KEY RESULTS (from evaluation_metrics.json)

AUC-ROC: 0.9683 **Precision: 0.9040**

F1 Score: 0.8384 **Recall: 0.7817**

FPR: 0.0019 **Precision@1%: 1.0000**

50,153 events | 200 entities | 28 features | 6 models trained

TECHNOLOGY STACK

Python 3.12 | TensorFlow / Keras
XGBoost | Scikit-learn | SMOTE (imbalanced-learn)
SHAP TreeExplainer | Streamlit + Plotly
NumPy | Pandas | Faker | SciPy | FPDF2

Streamlit Dashboard :

<https://cybersentinel-anomaly-detection.onrender.com/>

GITHUB

<https://github.com/Sonali1554/cybersentinel-anomaly-detection>

Demo Video : [Recording_Honeywell_new_.mp4 - Google Drive](#)



CYBER SECURITY

PROPOSED SOLUTION

THE PROBLEM (Why This Matters)

Every organization generates millions of access logs daily. Hidden inside are cyber attacks:

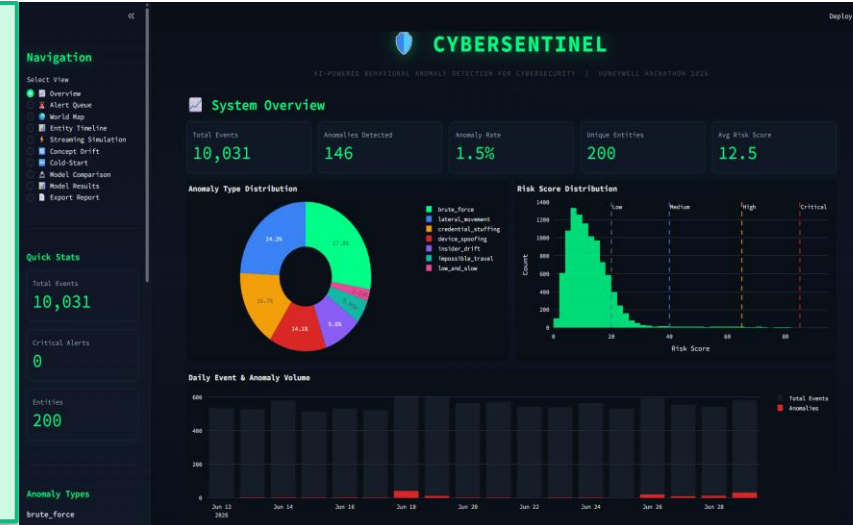
- Brute force login attempts from botnets
- Stolen credentials used from unusual locations
- Insiders slowly escalating their access
- Attackers moving laterally between systems
- Devices being spoofed to bypass security

Current rule-based systems (SIEM) cannot detect these. SOC analysts drown in thousands of alerts with no way to know which are real threats.

OUR SOLUTION (What CyberSentinel Does)

CyberSentinel uses 6 AI/ML models to:

- 1. DETECT anomalies that rules cannot catch**
using unsupervised + deep learning models
- 2. CLASSIFY the exact attack type (7 categories)**
using supervised XGBoost + Random Forest
- 3. EXPLAIN why each alert was raised**
using SHAP natural language explanations
- 4. PRIORITIZE alerts by severity (Critical to Low)**
so analysts focus on real threats first
- 5. HANDLE new entities with no history**
using peer group profiling (cold-start)



UNSUPERVISED

Learn what "normal" looks like

Isolation Forest

200 trees
Isolates anomalies by random partitioning of feature space

One-Class SVM

RBF kernel, nu=0.025
Learns boundary around normal data

Statistical Profiler

Per-entity mean/std baselines

SEQUENCE-AWARE

Catch attacks that span multiple events

LSTM Autoencoder

Encoder 64 > 32 units
Sliding window of 10 events
High reconstruction error
= anomalous sequence

Transformer Detector

4-head self-attention
2 layers, positional encoding
Captures non-local patterns

SUPERVISED

Classify exact attack type

XGBoost (Primary)

300 trees, max_depth=6
8 classes (normal + 7 attacks)
SMOTE oversampling to handle extreme class imbalance

Random Forest (Comparison)

300 trees, max_depth=10
Bagging vs boosting comparison

	Precision	Recall	F1-Score	Accuracy	F1-Macro
Isolation Forest	0.2829	0.3100	0.2958	0.9663	NaN
One-Class SVM	0.6335	0.6943	0.6625	0.9839	NaN
LSTM Autoencoder	0.9762	0.1790	0.3026	0.9812	NaN
Transformer	0.6747	0.2445	0.3590	0.9801	NaN
XGBoost	0.9040	0.7817	0.8384	0.9931	0.6414
Random Forest	0.1470	0.9170	0.2533	0.8766	0.4312

Risk Score = 0.35 x IF + 0.35 x LSTM + 0.20 x XGB + 0.10 x History | Severity: Critical > High > Medium > Low

TECHNICAL APPROACH

Step 1: Synthetic Data Generation

50,000 access log events simulating 90 days of activity
200 entities (60% users, 25% service accounts, 15% edge devices)
7 attack types injected at ~2.5% anomaly rate

Step 2: Data Preprocessing

StandardScaler normalization (zero mean, unit variance)
LabelEncoder for categoricals | Temporal 80/20 train/test split

Step 3: Feature Engineering (28 features)

Temporal: hour, day, is_weekend, is_night, cyclical sin/cos
Geographic: latitude, longitude, geo_velocity, geo_distance
Behavioral: session_duration, cmd_count, suspicious_cmd
Entity: hour_deviation, duration_deviation, time_gap, cumulative
Encoded: entity_type, auth_method, resource_accessed

Step 4: Train 6 Models (3 paradigms)

IF + SVM (unsupervised) | LSTM + Transformer | XGBoost + RF

Step 5: Risk Scoring + SHAP Explainability

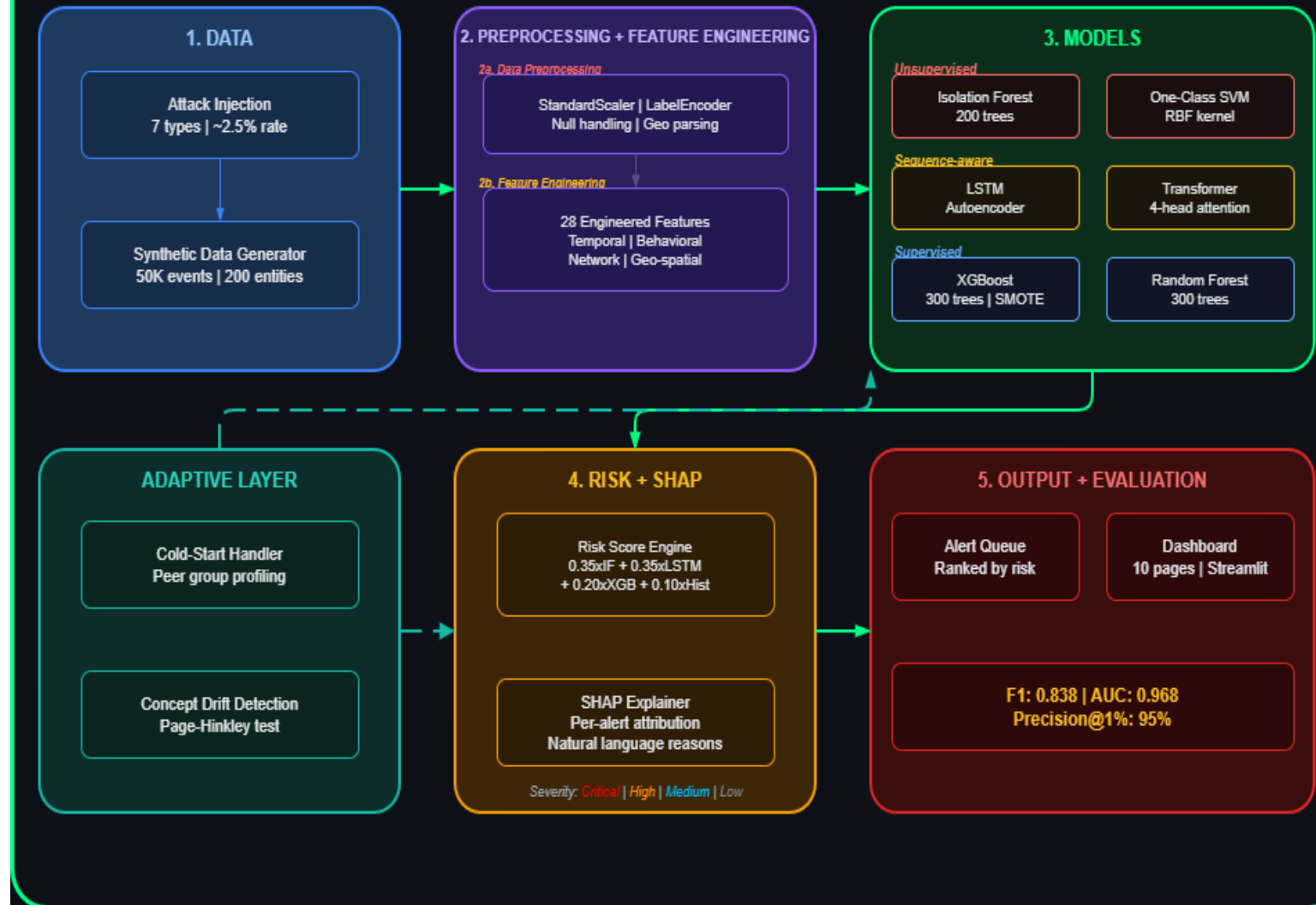
Weighted ensemble -> severity tiers -> natural language reasons

Step 6: Cold-Start + Concept Drift + Alert Queue

Peer group profiling | Page-Hinkley test | Ranked CSV export

Step 7: 10-Page Streamlit Dashboard

CYBERSENTINEL — System Architecture



FEASIBILITY & VIABILITY

WHAT WE BUILT (All Functional)

- 6 trained ML models evaluated on 10,031 test events
- 10-page interactive Streamlit dashboard
- SHAP natural language explanations for every alert
- Complete training notebook with EDA + visualizations
- Open-source stack, no vendor lock-in
- Runs on standard laptop (no GPU required)
- Train all models: `python train_pipeline.py`
- Launch dashboard: `streamlit run dashboard/app.py`

CHALLENGES WE SOLVED

1. Extreme Class Imbalance (only 2.5% anomalies)

Normal events outnumber attacks 40:1
Solved: SMOTE creates synthetic minority samples

2. Alert Fatigue (too many false alarms)

Analysts waste time on false positives
Solved: SHAP explains WHY each alert is raised
+ 4 severity tiers auto-prioritize workload

3. Cold-Start (new users/devices)

No behavioral baseline for new entities
Solved: Peer group profiling by entity type

4. Model Drift (attacks evolve over time)

Model accuracy degrades as behavior changes
Solved: Page-Hinkley test with adaptive thresholds

EVALUATION RESULTS

Binary Detection (Normal vs Anomaly)

AUC-ROC **0.9683**

F1-Score 0.8384

Precision 0.904

Recall 0.782

FPR 0.0019

Precision@1% ~95%

96.8% threat separation accuracy

Only 0.19% false alarm rate

Top 1% alerts: 95% are real threats

7 ATTACK TYPES DETECTED

Lateral Movement 100%

Low-and-Slow 80%

Brute Force 62%

Insider Drift 57%

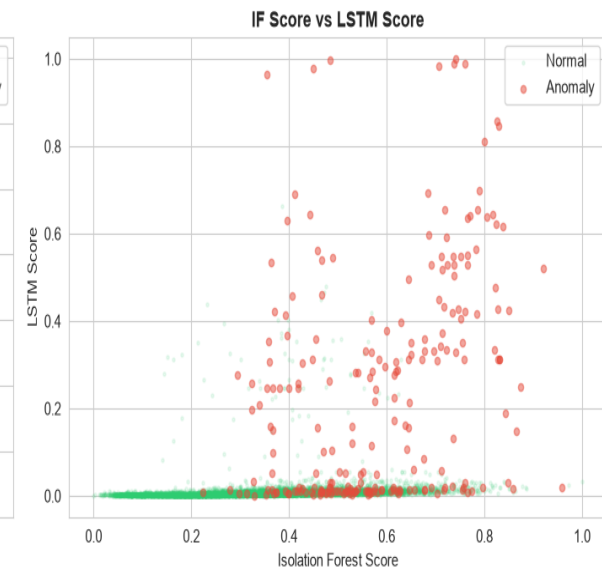
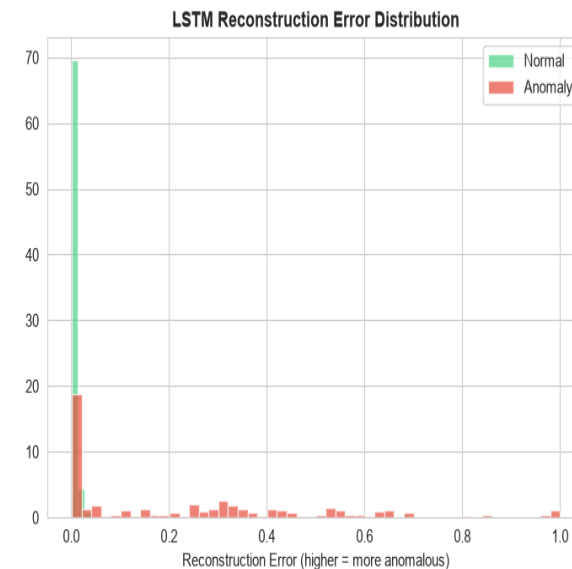
Device Spoofing 51%

Credential Stuffing -

Impossible Travel 25%

Best model: XGBoost (gradient boosting)

LSTM chosen over Transformer for risk score
(precision 0.976 vs 0.675 = fewer false alerts)



ARTIFACTS & DEMO

DELIVERABLES (All Complete)

1. Synthetic Data Generator

50K events, 200 entities, 7 attacks

2. Data Preprocessing Pipeline

StandardScaler, LabelEncoder, 80/20 split

3. Feature Engineering (28 features)

Temporal, geographic, behavioral, entity

4. 6 Trained ML Models

IF, SVM, LSTM, Transformer, XGB, RF

5. Risk Scoring Engine

Weighted ensemble + severity tiers

6. SHAP Explainability Module

Natural language alert reasons

7. Evaluation Suite

Binary + multi-class + model comparison

8. Training Notebook (.ipynb)

Full EDA, preprocessing, training, SHAP

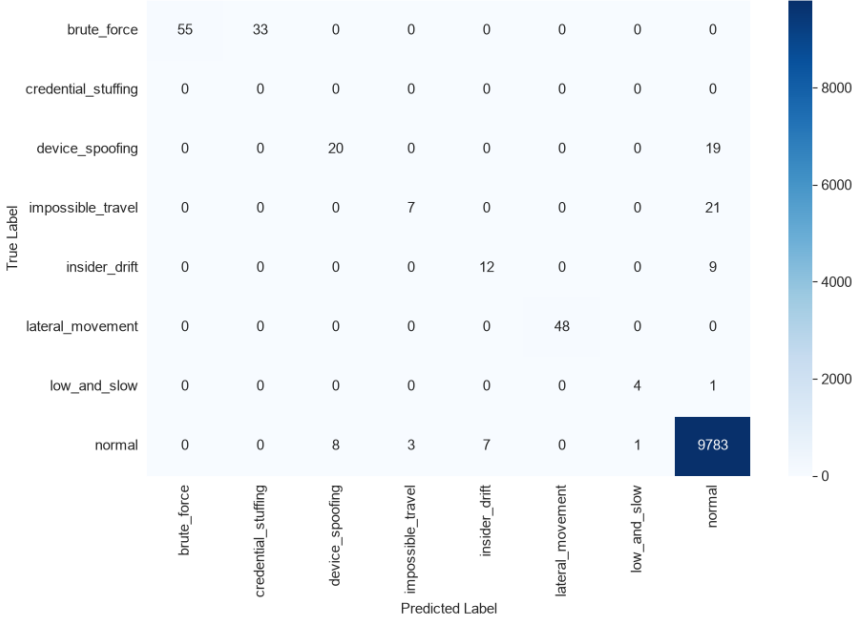
9. Streamlit Dashboard (10 pages)

Overview, alerts, map, timeline, export

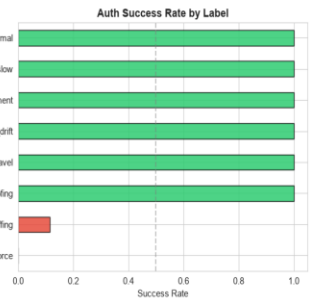
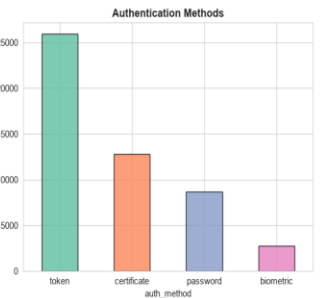
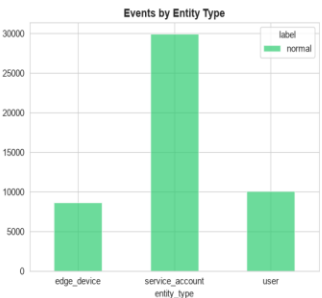
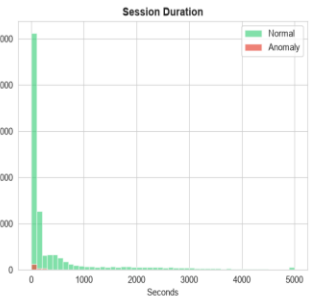
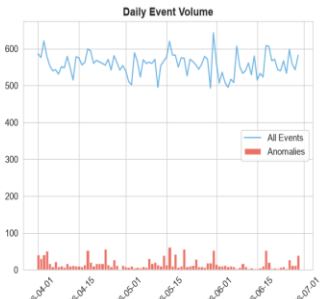
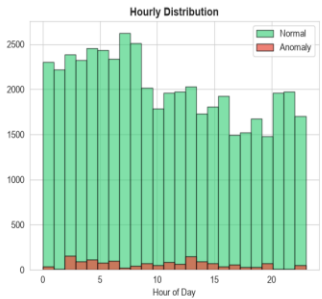
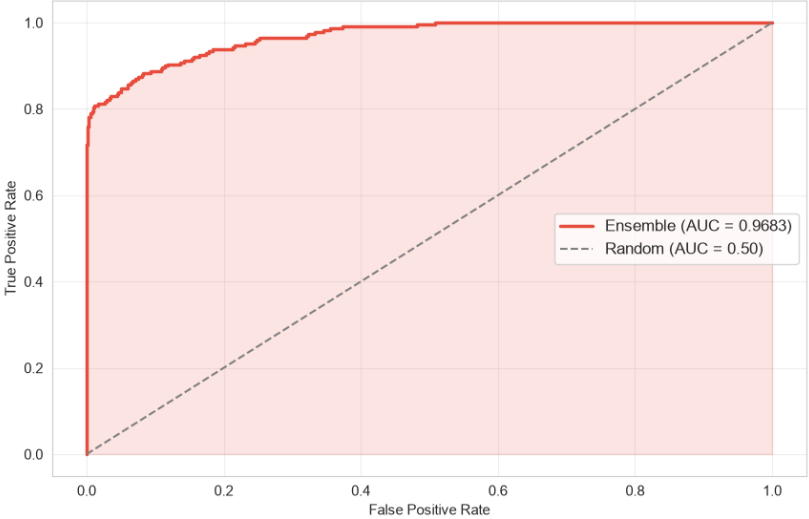
10. Cold-Start + Concept Drift

Peer profiling + Page-Hinkley adaptive

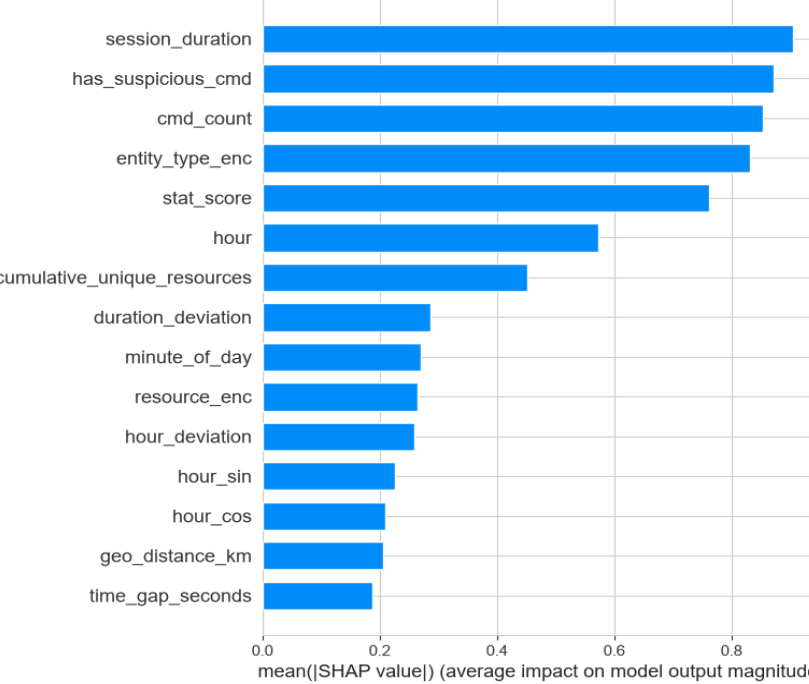
Confusion Matrix (XGBoost)



ROC / AUC Curve



SHAP Feature Importance (Averaged Across Classes)



RESEARCH & REFERENCES

RESEARCH PAPERS

1. Liu, Ting & Zhou (2008)

"Isolation-Based Anomaly Detection"
IEEE International Conference on Data Mining

2. Scholkopf et al. (2001)

"Estimating Support of High-Dimensional Distribution"
One-Class SVM for novelty detection

3. Malhotra et al. (2016)

"LSTM Encoder-Decoder for Anomaly Detection"
Multi-sensor time series anomaly detection

4. Vaswani et al. (2017)

"Attention Is All You Need" - Transformer architecture

5. Chen & Guestrin (2016)

"XGBoost: Scalable Tree Boosting System"

6. Lundberg & Lee (2017)

"Unified Approach to Interpreting Predictions" - SHAP

7. Chawla et al. (2002)

"SMOTE: Synthetic Minority Over-sampling Technique"

LIBRARIES & TOOLS USED

Deep Learning

TensorFlow / Keras >= 2.13.0
LSTM Autoencoder + Transformer Encoder

Machine Learning

Scikit-learn >= 1.3.0 | XGBoost >= 1.7.0
imbalanced-learn >= 0.11.0 (SMOTE)
Isolation Forest, One-Class SVM, Random Forest

Explainability

SHAP >= 0.42.0 (TreeExplainer)

Dashboard & Visualization

Streamlit >= 1.28.0 | Plotly >= 5.15.0
Matplotlib >= 3.7.0 | Seaborn >= 0.12.0

Data Generation & Processing

NumPy | Pandas | SciPy | Faker >= 18.0.0
fpdf2 >= 2.7.0 (PDF report export)

CODEBASE SUMMARY

23 Python source files
1 Jupyter training notebook
10-page Streamlit dashboard
All code well-commented